PROJECT PRESENTATION

PACMAN



UNIVERSITÉ CÔTE D'AZUR

Generality

A maze is a grid of cells with walls. An algorithm “generates” a maze by removing walls to make corridors. Different algorithms remove walls in different patterns → different “maze styles”. For Pac-Man, we want multiple routes (loops), symmetry, teleporters, and a ghost room.

“Remove a wall” = create a corridor.

“Connected” means you can reach everywhere.

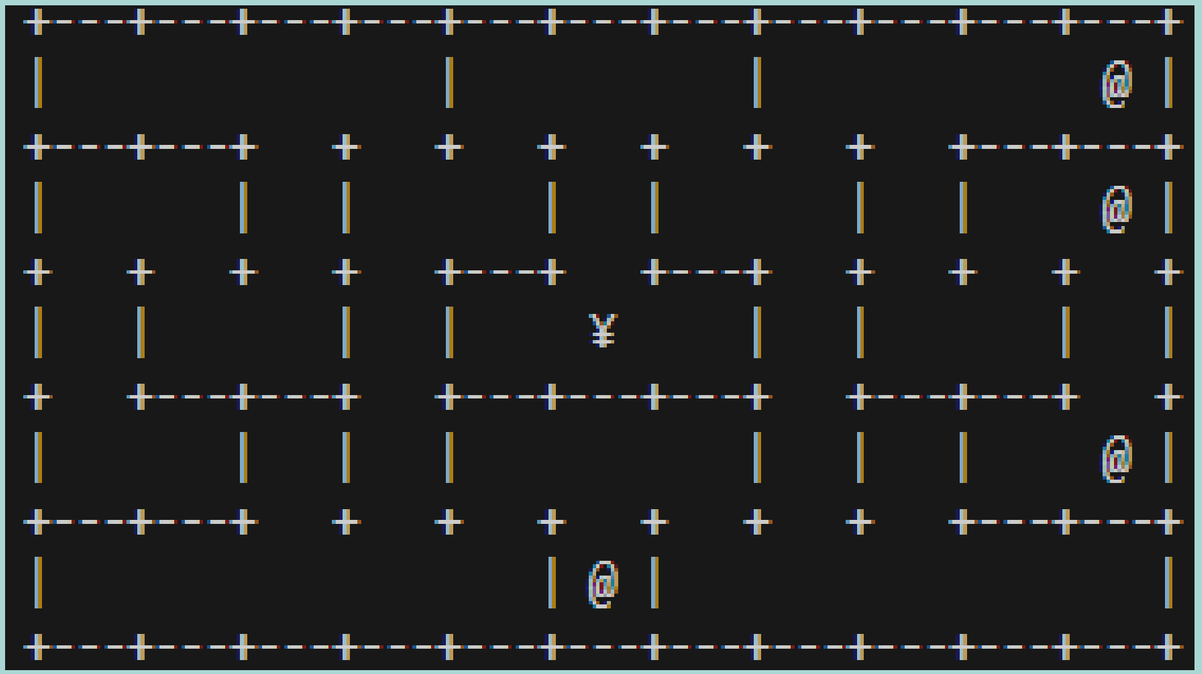
“Pipeline” = steps we apply to make it Pac-Man-like.

Pac-Man-Style Maze Generation Algorithms

- We model the maze as a grid: removing a wall creates a passage.
- Goal: a connected, playable maze (multiple routes, not too many dead ends).
- Pipeline: generation → add loops (cycles and extra connection) → Pac-Man-style symmetry → teleporters.

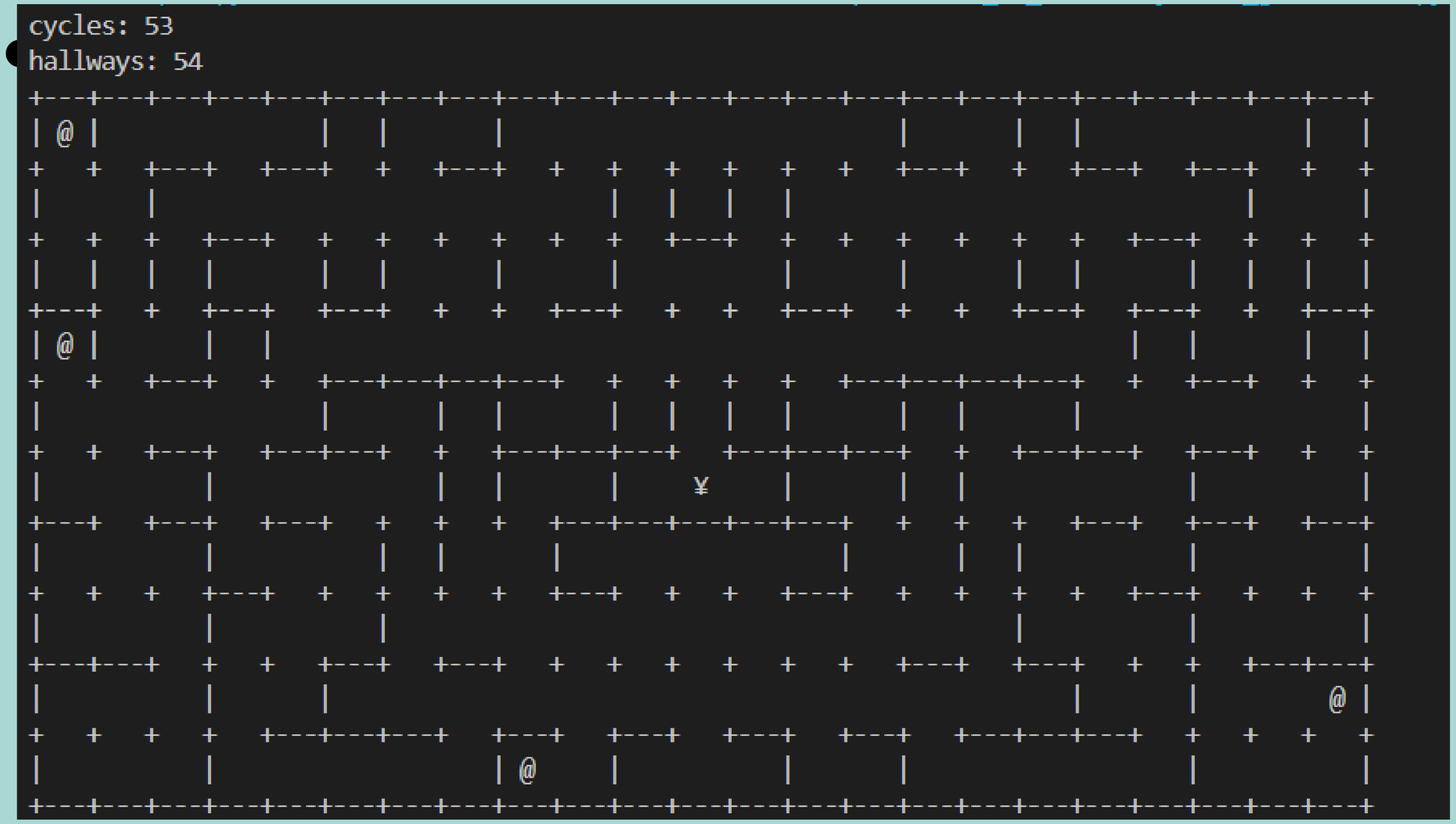
4 algorithms, 4 “styles”

- DFS: carves deep first → long corridor many dead ends
- Prim: expands (step by step) via a frontier → more uniform, more evenly distributed
- Kruskal : removes random walls (connects 2 different areas) + Union-Find → balanced overall structure



DFS

– Tetris (growing-tree): “active cell” list → then expands dense, varied maze



Tetris

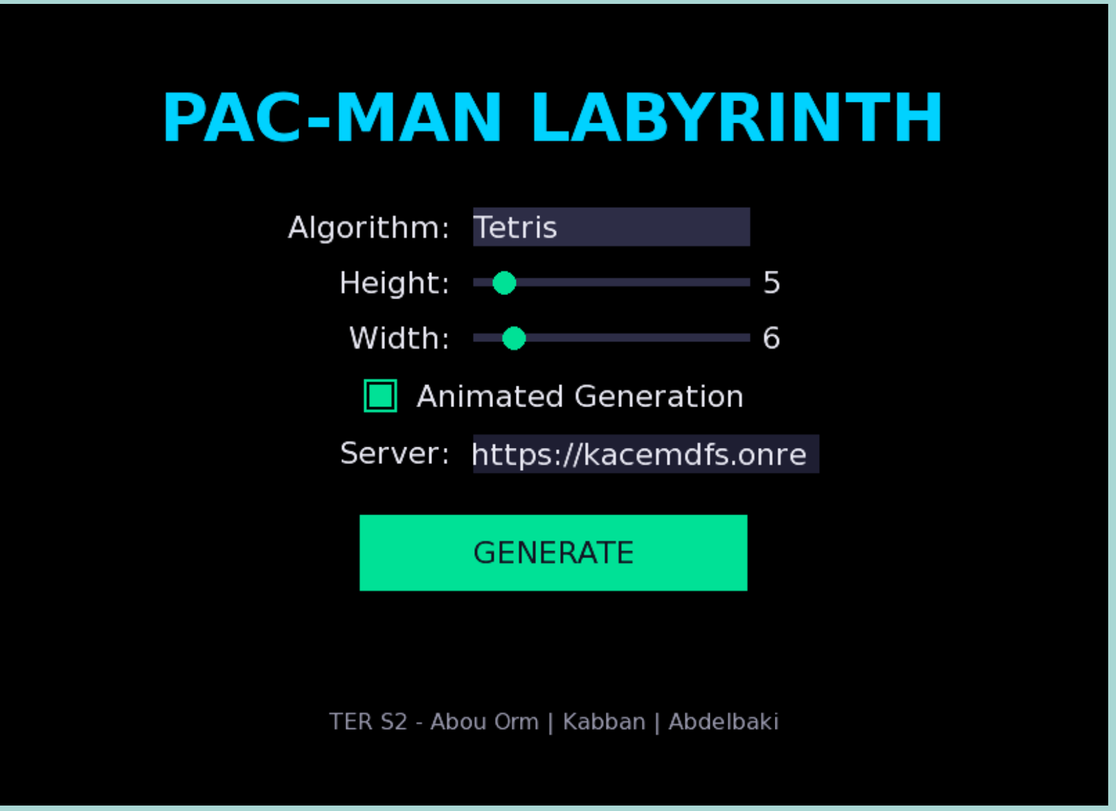
Why It’s Well-Suited for Pac-Man

- Loops = more than one way to go (better chases).
- We add loops (cycles) → multiple paths, more interesting chases
- Symmetry + “ghost room” + teleporters → classic Pac-Man feel

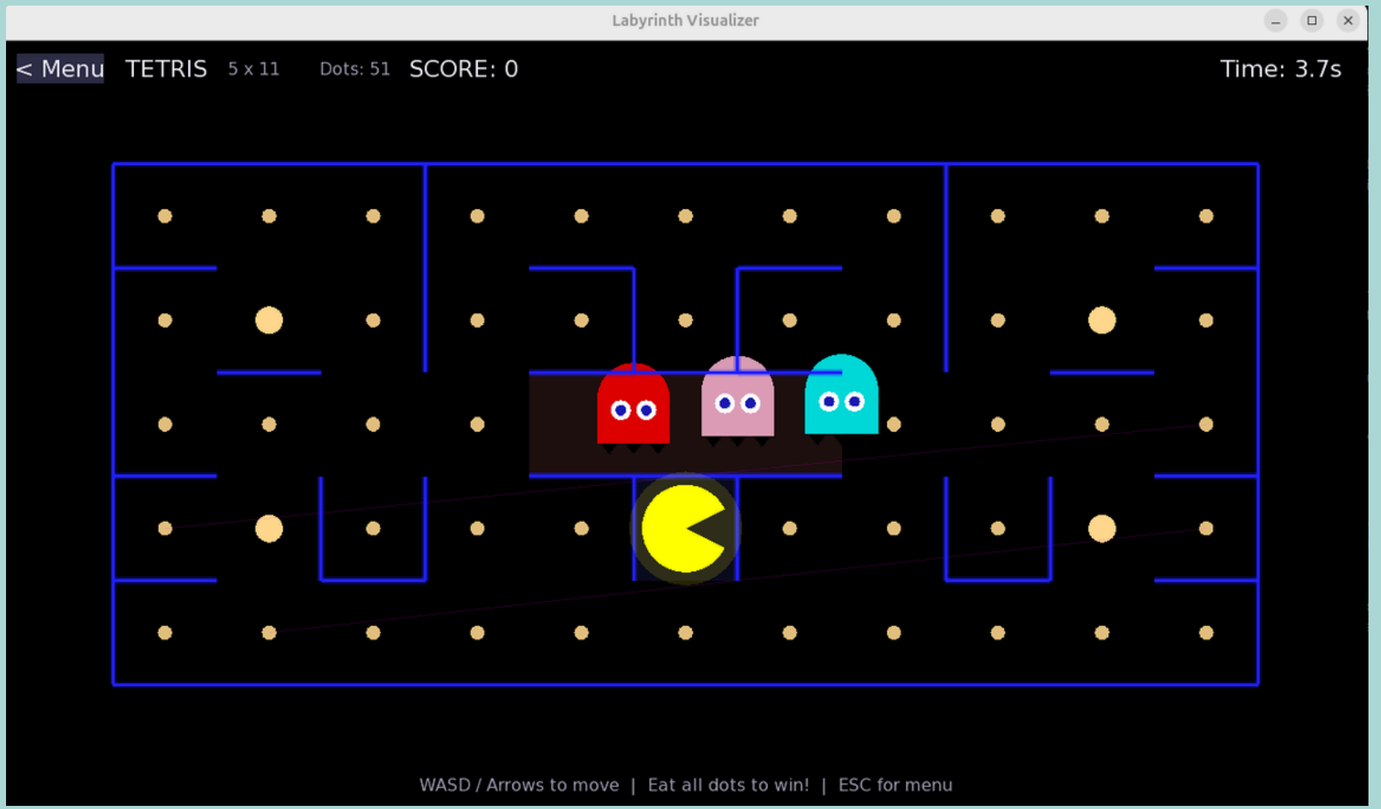
Graphical Interface (LibGDX)

- Built the desktop graphical client in Java (LibGDX + LWJGL3)
- **Goal**: visualize generated labyrinths in a Pac-Man style and make them playable
- Integrated rendering, UI, animation, and gameplay into one coherent interface
- Connects to the Flask API to fetch maze JSON (server configurable)

UI / Screens / UX Flow



Menu screen



Maze screen

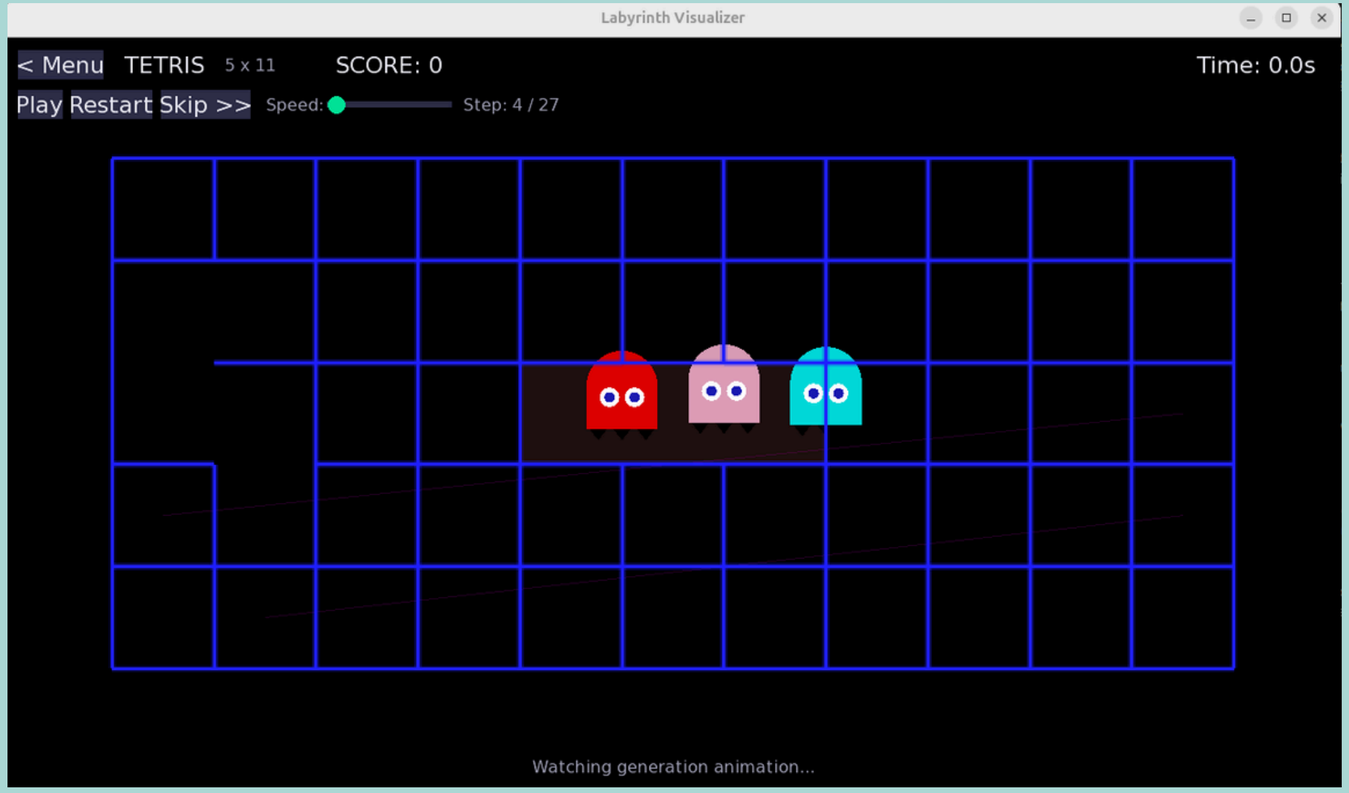
- HUD overlay: score, dots remaining, timer, status messages
- Inputs: WASD/arrow movement + ESC back to menu

API Connection (Frontend ⇔ Backend)

- User chooses algorithm + size + “Animated generation”
- Frontend sends GET /labyrinth to the server address
- Receives JSON maze data (grid + optional steps)
- If request fails → show error message in the interface

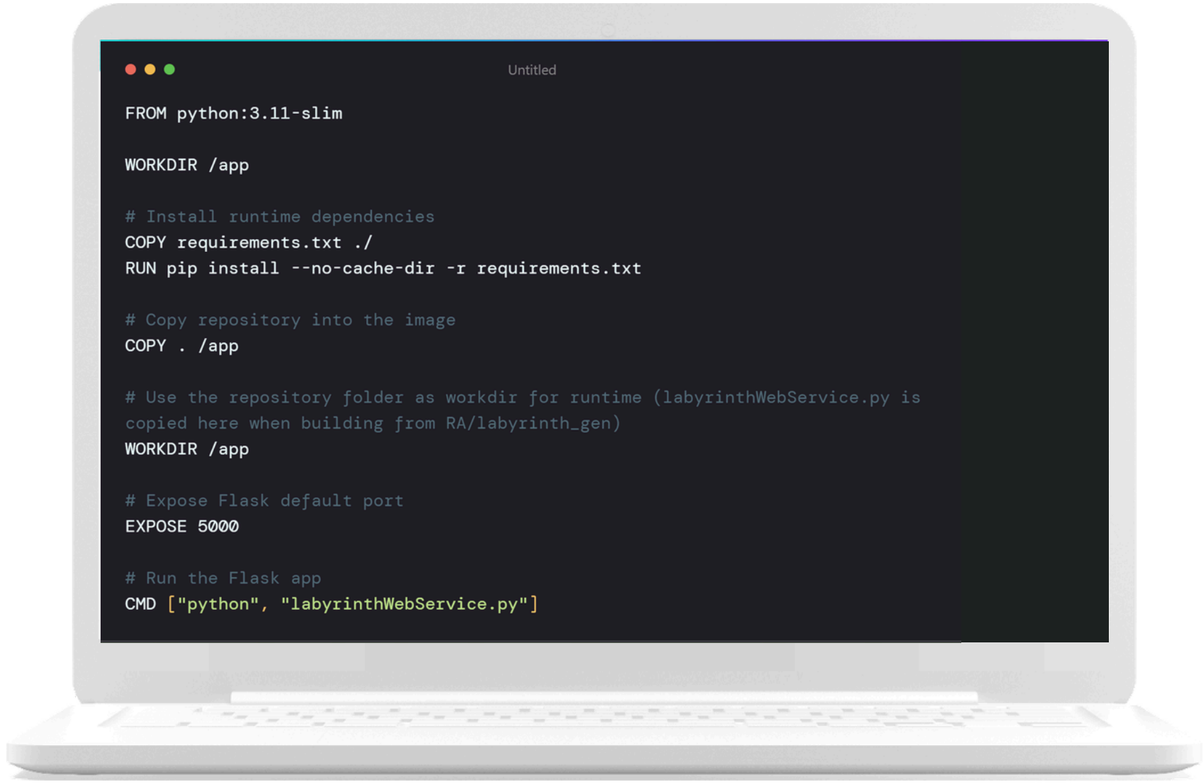
From JSON to Screen

- Convert JSON grid
- The maze builds progressively on screen
- The backend sends a list of steps (wall removals)
- HUD controls for the user
- When the animation finishes, the final maze stays and you can play normally



Docker Containerisation

`python:3.11-slim`	Minimal OS footprint — no unnecessary packages
`requirements.txt first`	Docker layer cache — deps only reinstall on change
`--no-cache-dir`	Further reduces final image size
`EXPOSE 5000`	Required by Render to route public traffic
`CMD python …`	Direct Flask execution (not flask run) for compatibility



Core Testing

test_generators.py

Functional tests for maze generation logic.

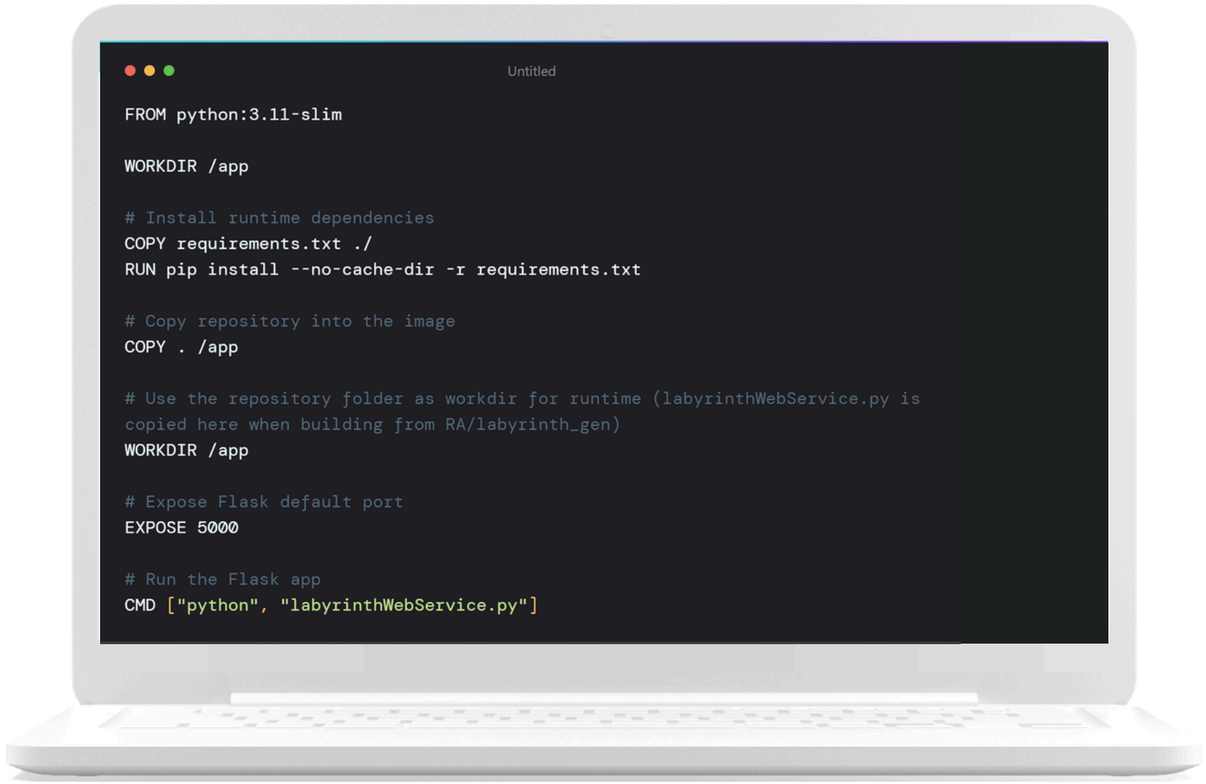
- **Dimension Verification:** Confirms generated maze matches requested size.
- **Cell Structure:** Validates cell properties (walls, ghost, TP) and data types.
- **Teleportation Pairs:** Checks for symmetric, bidirectional teleportation links.
- **Ghost Room Presence:** Ensures ghost cells are correctly generated.
- **Step Recording:** Verifies optional generation history for visualization.

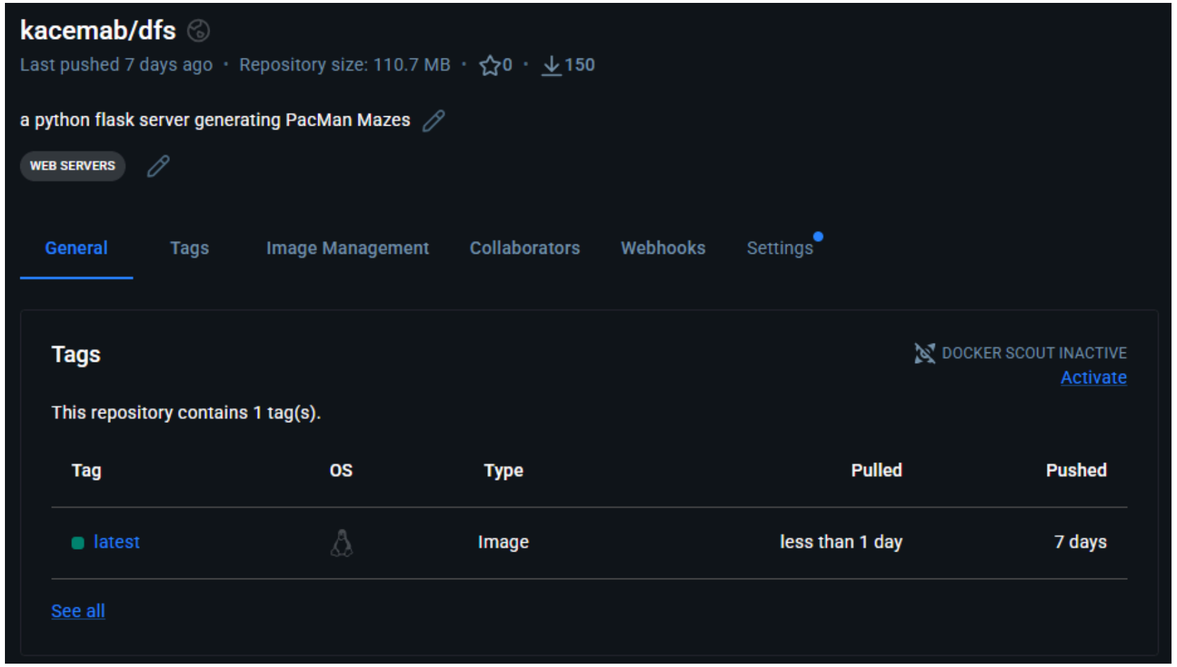
test_maze_properties.py

Structural validation of maze topology.

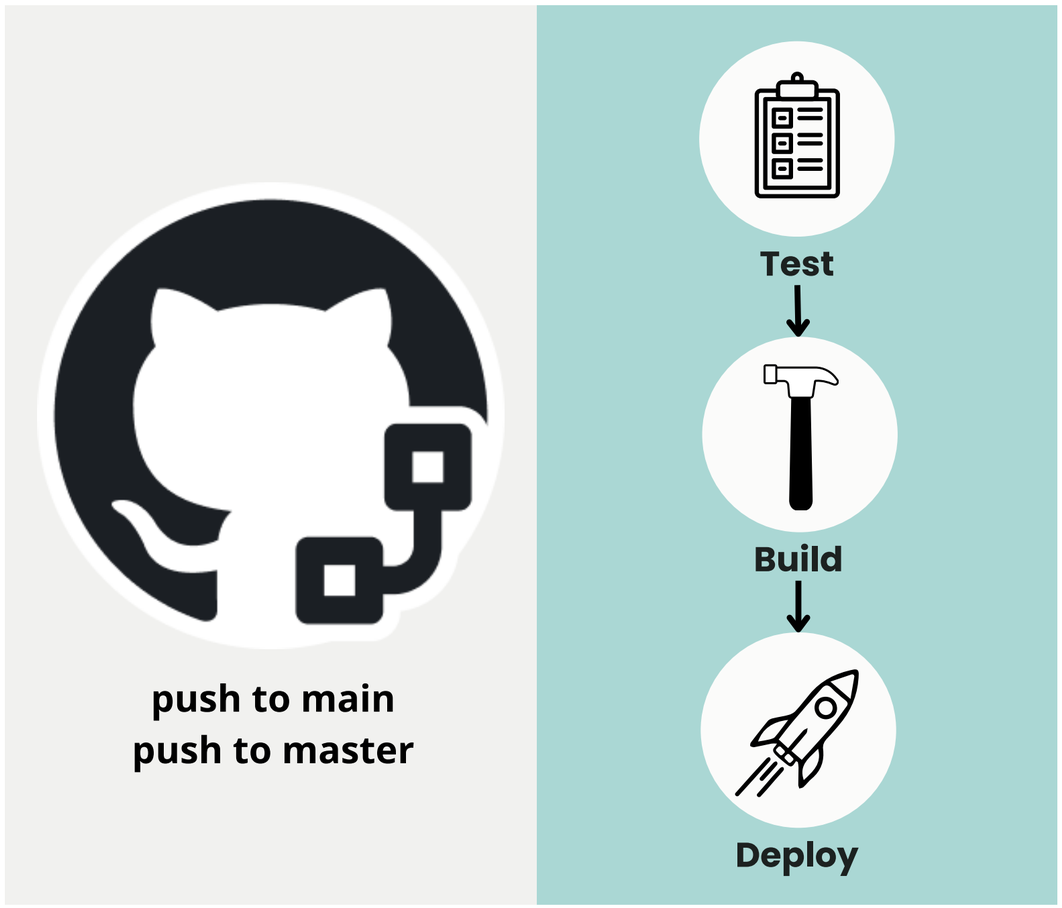
- **Connectivity Validation:** BFS flood-fill ensures all non-ghost cells are reachable.
- **Cycle Detection:** Uses graph theory to verify maze complexity and absence of isolated sections.
- **Wall Symmetry Checks:** Confirms correct wall placement, accounting for specific design elements like the Pac-Man opening.

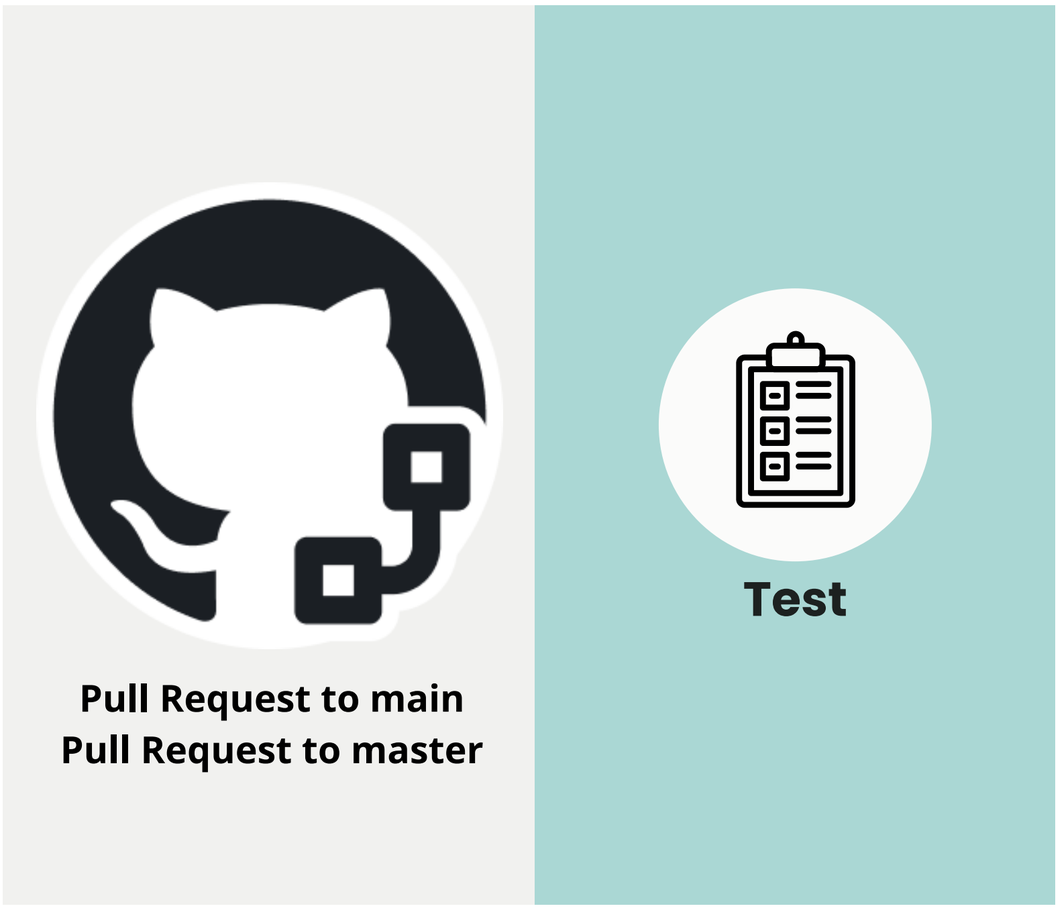
Docker Containerisation





CI/CD Pipeline - GitHub Actions





Cloud Deployment – Render



Render watches for deploy hooks (HTTP calls from GitHub Actions).

Cloud Deployment – Render

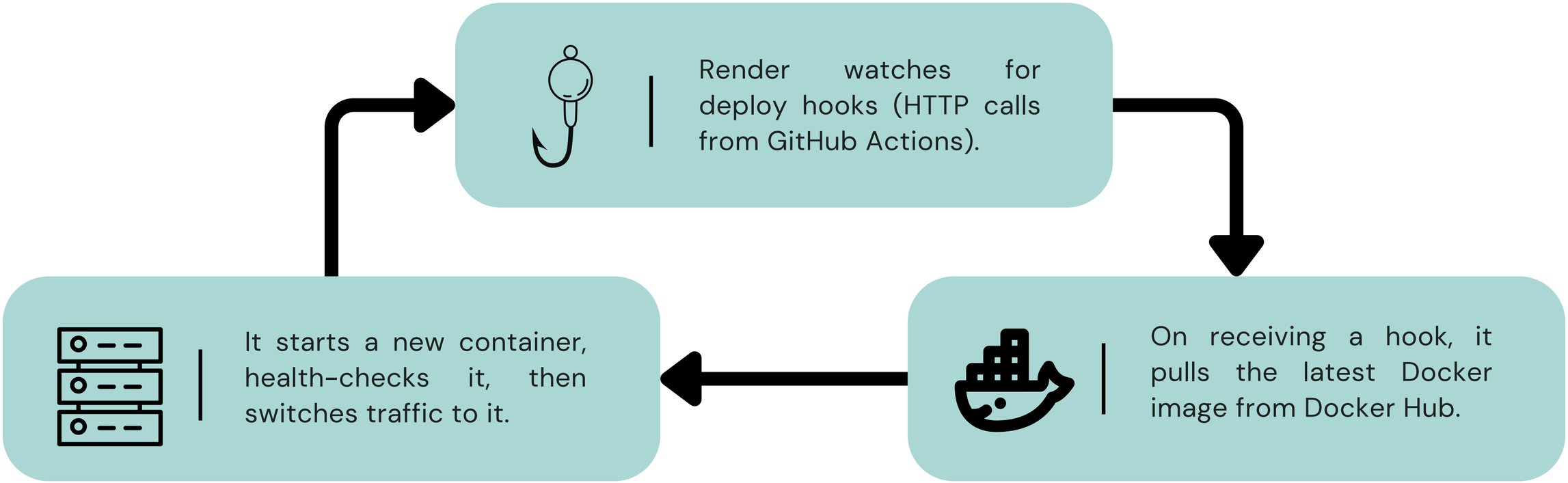


Render watches for deploy hooks (HTTP calls from GitHub Actions).



On receiving a hook, it pulls the latest Docker image from Docker Hub.

Cloud Deployment – Render



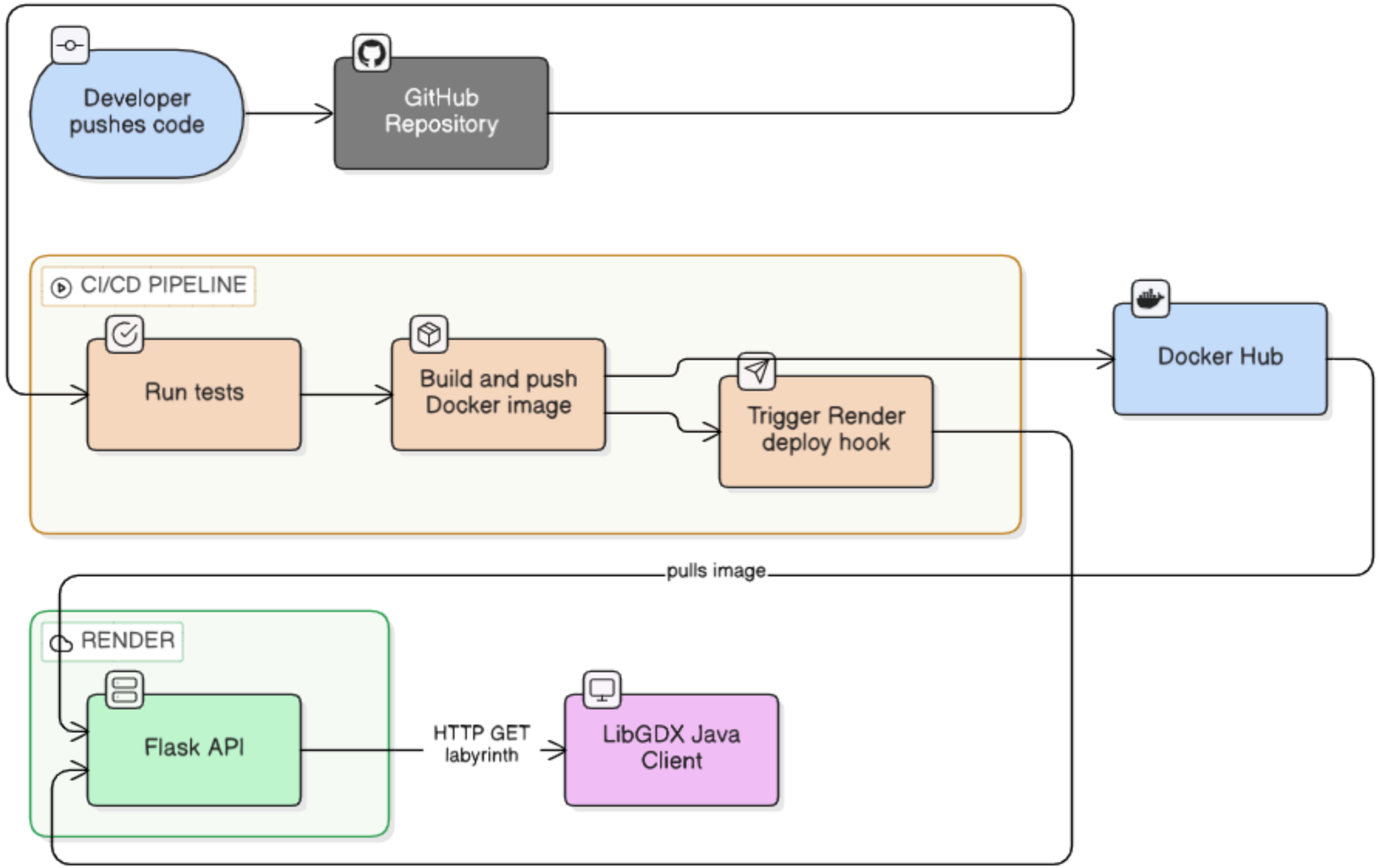
```
{
  "algorithm": "dfs",
  "height": 5,
  "width": 6,
  "rows": [
    [
      {
        "x": 0,
        "y": 0,
        "walls": [true, false, true, true],
        "isGhost": false,
        "isTP": false,
        "TP": null
      }
    ]
  ]
}
```


https://pacmanS2G.onrender.com/labyrinth?algorithm=DFS&height=15&width=15"

API Reference

Parameter	Values	Description
“algorithm”	dfs, tetris	Maze generation algorithm
“height”	3 – 50	Number of rows
“width”	3 – 50	Number of columns

GLOBAL ARCHITECTURE



Thank you very much!